

Week 10: LLM Pipelines

Python 200 I Mentor Session

What This Week Covers

In weeks 5–7, the LLM was the end product — students sent messages and got responses. This week, the framing shifts: an LLM is a processing step inside a larger pipeline.

- **The double-transform pattern** — ML classifier first, LLM enrichment second
- **ML vs LLM** — which tool belongs where in a pipeline, and why
- **ML inference** — read `weather_raw`, run the Week 4 classifier, produce predictions
- **LLM enrichment** — turn structured predictions into human-readable recommendations
- **Write to `weather_enriched`** — complete the second half of the ETL pipeline

By the end of the week, `weather_enriched` has one row per day: a binary prediction, a confidence score, and a one-sentence recommendation written by an LLM.

ML vs LLM in a Pipeline

Both ML models and LLMs belong in the **Transform** step of an ETL pipeline. The question is which one to reach for.

Task	Right tool	Why
Binary classification on numeric features	ML model	Fast, deterministic, cheap, reproducible
Natural-language summary or recommendation	LLM	Requires language generation and judgment
Arithmetic, filtering, aggregation	Code	LLMs are slow, expensive, and wrong for math
Classifying freeform text	LLM	Requires reading comprehension

The weather classifier: four numeric features → binary output. ML is the right tool — faster, cheaper, and more reproducible than asking an LLM "is this a good running day?"

The recommendation: "Great conditions — mild temperatures, no rain, and light wind." That requires language generation. ML can't do it.

The Double-Transform

Two transforms. Two different tools. Each where it fits best.

Transform 1 — ML classifier:

- Input: weather features from `weather_raw`
- Output: `good_for_running` (bool) + `confidence` (float)
- Cost: essentially free; runs in milliseconds for 365 records

Transform 2 — LLM:

- Input: weather features + ML prediction + confidence
- Output: `llm_summary` (one sentence)
- Cost: ~\$0.02 for 365 records at `gpt-4o-mini` pricing; a few minutes wall-clock

The ML model cannot explain its prediction in plain language. The LLM cannot reliably classify four numeric features. Together they produce something neither could alone.

Discussion: Could you skip the ML step and just ask the LLM to classify every record directly? What are the tradeoffs?

The Saved Pipeline as a Component

In Week 4, students saved the best GridSearchCV result to disk:

```
joblib.dump(best_pipe, "models/weather_classifier.pkl")
```

In Week 10, that file is loaded and used as a pipeline component:

```
import joblib  
  
clf = joblib.load("models/weather_classifier.pkl")
```

`clf` is immediately ready to call `.predict()` and `.predict_proba()`. No retraining, no re-fitting. This is the payoff of model persistence: the training cost is paid once; the artifact can be used anywhere the `.pkl` is available.

The model expects features in a specific order. Load that order from the metadata file — don't assume column order from the database.

Lesson 1 Demo: Double-Transform Pattern

Open: Lesson 1 → double-transform pattern

Focus on	Skim for now
The ML vs LLM decision table — when each tool belongs	LLM pricing specifics (changes frequently)
The saved Pipeline as a reusable component — load once, call <code>.predict()</code>	Batching and concurrency strategies for large datasets
Designing constrained prompts for pipeline use	OpenAI Batch API

Ask: where in the pipeline does the ML step run? Where does the LLM step run? What would happen if you ran them in the wrong order?

Discussion: The lesson says to use a unit test as a heuristic: if you can write one with a single expected output, use code. Does the weather classification pass that test? Does the running recommendation?

Reading from weather_raw

```
response = supabase.table("weather_raw").select("*").execute()  
raw_rows = response.data # list of dicts
```

Before classifying everything, check which dates are already enriched:

```
enriched_response = supabase.table("weather_enriched").select("date").execute()  
already_done = {row["date"] for row in enriched_response.data}  
  
to_classify = [row for row in raw_rows if row["date"] not in already_done]  
print(f"Records to classify: {len(to_classify)}")
```

This is the **incremental processing** pattern: only do work that hasn't already been done. On the first run, all 365 records are classified. On re-runs, only new records are processed.

Especially important for LLM calls — re-processing records you've already paid for is waste.

Building the Feature DataFrame

The saved Pipeline expects features in the order it was trained on. Get that order from the metadata file:

```
import json, pandas as pd

with open("models/weather_classifier_metadata.json") as f:
    metadata = json.load(f)

FEATURES = metadata["features"]
# ['temperature_2m_max', 'temperature_2m_min', 'precipitation_sum', 'wind_speed_10m_max']

df = pd.DataFrame(to_classify)
X = df[FEATURES] # exact columns, exact order
```

`df[FEATURES]` rather than `df.drop(["date", "loaded_at"])` — explicit about what the model expects, robust to new columns added to the table later.

Discussion: What would go wrong if you passed `df.drop(["date", "loaded_at"], axis=1)` and the database returned columns in a different order?

Running the Classifier

```
predictions = clf.predict(X)          # array of 0s and 1s
probabilities = clf.predict_proba(X)[: , 1] # probability of class 1

print(f"Good days predicted: {predictions.sum()} / {len(predictions)}")
```

`predict_proba` returns a 2D array: column 0 is P(skip), column 1 is P(good for running). `[: , 1]` extracts the confidence score.

Build the enrichment records — note the type conversions:

```
enrichment_records = [
    {
        "date": to_classify[i]["date"],
        "good_for_running": bool(predictions[i]), # numpy int → Python bool
        "confidence": round(float(probabilities[i]), 4),
        "llm_summary": None, # filled in next step
    }
    for i in range(len(to_classify))
]
```

Lesson 2 Demo: ML Inference

Open: Lesson 2 → ML inference on database records

Focus on	Skim for now
Incremental processing check — <code>already_done</code> set, filter on <code>to_classify</code>	<code>predict_proba</code> internals
<code>df[FEATURES]</code> — explicit column selection in training order	Sanity-checking output fractions
<code>bool()</code> and <code>float()</code> conversions — numpy types vs Python types	The borderline prediction analysis at the end of the lesson

Run the classifier on the loaded `weather_raw` data. Check that ~30–40% of days are classified as good (consistent with Charlotte, NC, 2023 data).

Discussion: `already_done` has 200 entries and `weather_raw` has 365. How many records does `to_classify` contain? What happens to the other 200?

Designing the LLM Prompt

Interactive LLM use rewards open-ended prompts. **Pipeline use rewards constrained ones.**

```
SYSTEM_PROMPT = (  
    "You are writing a one-sentence running recommendation for a daily weather summary app. "  
    "You will receive weather conditions for a single day and a machine learning prediction. "  
    "Write exactly one sentence – direct, practical, and specific to the conditions. "  
    "Do not use bullet points, headers, or phrases like 'Based on the data'."   
)  
  
def make_user_message(row, good_for_running, confidence):  
    prediction_text = "good for running" if good_for_running else "not ideal for running"  
    return (  
        f>Date: {row['date']}\n"  
        f"High: {row['temperature_2m_max']}°C, Low: {row['temperature_2m_min']}°C\n"  
        f"Precipitation: {row['precipitation_sum']} mm\n"  
        f"Max wind speed: {row['wind_speed_10m_max']} km/h\n"  
        f"Model prediction: {prediction_text} (confidence: {confidence:.0%})"  
    )
```

Passing confidence to the LLM is intentional — a borderline prediction is worth flagging.

LLM Enrichment + Writing Results

```
for i, record in enumerate(enrichment_records):
    raw_row = to_classify[i]
    try:
        response = client.chat.completions.create(
            model="gpt-4o-mini",
            messages=[
                {"role": "system", "content": SYSTEM_PROMPT},
                {"role": "user", "content": make_user_message(
                    raw_row, record["good_for_running"], record["confidence"])}],
            max_tokens=100,
        )
        record["llm_summary"] = response.choices[0].message.content.strip()
    except Exception as e:
        print(f" API error on {record['date']}: {e}")
        record["llm_summary"] = "Recommendation unavailable."

# Write to weather_enriched
supabase.table("weather_enriched").upsert(enrichment_records, on_conflict="date").execute()
```

Catch exceptions per record — one API error should not crash the entire pipeline.

Lesson 3 Demo: LLM Enrichment

Open: Lesson 3 → LLM enrichment and writing to weather_enriched

Focus on	Skim for now
The system prompt — one-sentence constraint, no bullet points	<code>validate_summary()</code> heuristic function
Passing ML confidence to the LLM prompt — why it's useful context	<code>max_tokens=100</code> and token sizing
<code>try/except</code> per record — fallback string vs crashing	Spot-check queries at the end of the lesson

Run the complete double-transform script. Check `weather_enriched` in the dashboard — 365 rows, each with a binary label, confidence score, and one-sentence recommendation.

Discussion: The pipeline uses `"Recommendation unavailable."` as a fallback for API errors. What are the tradeoffs? How would you find and re-process failed records later?

Week 10 Wrap-Up

- **Double-transform** — ML classifier (deterministic, cheap) first; LLM (language generation) second; each where it fits best
- **Incremental processing** — check `already_done` before classifying; only process new records; saves money on LLM calls
- `df[FEATURES]` — always select features explicitly, in training order; don't rely on column order from the database
- **Constrained prompts** — pipeline LLM calls need tight constraints: one sentence, no bullet points, no hedging; easier to validate
- `try/except` **per record** — one API failure should not crash the pipeline; fallback values keep the run going
- **Upsert** — idempotent writes to `weather_enriched`; safe to re-run

Takeaway: An LLM in a pipeline is not a chatbot — it's a transform step with inputs and expected output format. Constraining it to produce parseable output, handling failures per record, and only calling it on new data are the patterns that make it production-ready. In Week 11, Prefect wraps all of this into an orchestrated, observable flow.